Python Programming Language Reference

Comprehensive Core Concepts, Memory Internals, OOP, Async & Ecosystem Reference for RAG Ingestion

LANGUAGE CORE | DATA STRUCTURES | OOP & MAGIC METHODS | ASYNC & CONCURRENCY | RAG KNOWLEDGE BASE

1. Overview & Execution Architecture

Python is a high-level, interpreted, dynamically-typed, garbage-collected programming language created by Guido van Rossum (1991). Key runtime characteristics:

- **Interpreter Pipeline:** Source code (`.py`) → Lexer/Parser → Abstract Syntax Tree (AST) → Compiler → Bytecode (`.pyc`) → CPython Virtual Machine (PVM).
- **Memory Management:** Managed via Reference Counting and a Generational Garbage Collector (detects cyclical references across 3 generations: Gen 0, Gen 1, Gen 2).
- **Global Interpreter Lock (GIL):** A mutual-exclusion lock in CPython ensuring only one native thread executes Python bytecode at a time. Prevents true multi-threaded CPU-bound parallelism, making `multiprocessing` or C-extensions necessary for heavy computing.

2. Built-in Data Types & Time Complexity

Data Type	Mutability	Internal Structure	Access / Search Time	Common Use Case
`int` / `float` / `bool`	Immutable	Primitive Object Wrappers	O(1)	Scalar values & logical flags
`str`	Immutable	Array of Unicode code points	Index: O(1) / Search: O(n)	Text data processing
`list`	Mutable	Dynamic Array of Pointers	Index: O(1) / Search: O(n) / Append: O(1) amortized	Ordered collection of elements
`tuple`	Immutable	Fixed-size Array of Pointers	Index: O(1) / Search: O(n)	Read-only sequences, dictionary keys
`dict`	Mutable	Hash Table with Open Addressing	Lookup / Insert / Delete: O(1) avg	Key-Value maps, fast lookups
`set`	Mutable	Hash Table (Keys only)	Add / Remove / Lookup: O(1) avg	Unique items, set operations (&, |, -)

List & Dict Comprehensions

```
# List Comprehension with filtering evens_squared = [x**2 for x in range(10) if x % 2 == 0] # Dictionary Comprehension char_counts = {char: text.count(char) for char in set("rag_system")}
```

3. Control Flow & Advanced Functions

Functions in Python are **first-class objects**—they can be passed as arguments, returned from other functions, and assigned to variables.

Variable Arguments (*args, **kwargs) & Decorators

```
def log_execution(func): def wrapper(*args, **kwargs): print(f"Executing {func.__name__}...") result = func(*args, **kwargs) return result return wrapper @log_execution def calculate_similarity(v1, v2): return sum(a * b for a, b in zip(v1, v2))
```

Generators (`yield`) for Memory Efficiency

Generators compute values lazily using iterators, preserving low RAM footprints when processing streaming data or massive datasets in RAG pipelines.

```
def stream_chunks(file_path): with open(file_path, 'r') as f: for line in f: yield line.strip() # Memory footprint stays minimal
```

4. Object-Oriented Programming (OOP) & Dunder Methods

Python supports full Object-Oriented Programming including encapsulation, inheritance, method overriding, and duck typing.

```
class VectorStore: def __init__(self, collection_name: str): self.name = collection_name self._vectors = {} # Protected attribute def add_vector(self, doc_id: str, embedding: list): self._vectors[doc_id] = embedding def __len__(self): # Dunder method for len(instance) return len(self._vectors) def __getitem__(self, doc_id): # Subscript access store[doc_id] return self._vectors.get(doc_id)
```

5. Exception Handling & Context Managers

Robust error handling prevents system crashes during standard pipeline operations (e.g. API failures, missing files).

```
try: response = llm.invoke(prompt) except ConnectionError as e: print(f"Network error during LLM invocation: {e}") except Exception as e: print(f"Unexpected error: {e}") finally: cleanup_resources()
```

> **Context Managers (`with` statement):** Ensures resource cleanups (like closing database connections or files) occur automatically via `__enter__` and `__exit__` protocols.

6. Concurrency & Parallelism Model

Module	Model	Best Used For	GIL Status
`asyncio`	Single-threaded Event Loop (Cooperative)	High-volume I/O, Web Scraping, Network requests	Runs on 1 thread
`threading`	Preemptive OS Threads	I/O-bound tasks (file reads, database queries)	Locked by GIL
`multiprocessing`	Separate Python Processes	CPU-bound tasks (Vector embeddings, Matrix math)	Bypasses GIL

Async Code Sample (`asyncio`)

```
import asyncio async def fetch_embedding(text): await asyncio.sleep(0.1) # Simulate I/O latency
return [0.1, 0.2, 0.3] async def main(): tasks = [fetch_embedding(t) for t in ["doc1", "doc2",
"doc3"]] results = await asyncio.gather(*tasks) return results
```

7. Standard Library & Modern Enhancements

- **Type Hinting (PEP 484):** `def query(text: str, k: int = 4) -> list[dict]:` improves IDE autocompletion and static analysis (via `mypy`).
- **Dataclasses:** `@dataclass` decorator reduces boilerplate for data container objects.
- **Match-Case (Python 3.10+):** Structural pattern matching for clean decision logic.
- **Key Built-in Modules:** `math`, `os`, `pathlib`, `json`, `itertools`, `functools`, `collections` (`defaultdict`, `Counter`), `re` (Regex).